COMPILER CONSTRUCTION LAB · PA1 · GROUP 5

Q&A Preparation

60 questions the instructors and classmates are likely to ask after our talk, with answers you can say out loud and the place in the code to point at.

How to answer: answer first → one sentence of "why" → point to the code. 20–40 seconds. If you don't know: say what you'd expect and offer to check it live.

A. The big picture (1–5)
B. Indentation (6–15)
C. Other lexer questions (16–24)
D. Grammar and precedence (25–36)
E. Source locations (37–41)
F. Error handling and recovery (42–51)
G. Testing (52–55)
H. Team, process, and what comes next (56–60)

A. The big picture

Q1 What was the hardest part of PA1 for you?

Getting a parser that *accepts* ChocoPy was not the hard part. Matching the reference's AST **exactly, every line and column**, was. The locations follow conventions that no document describes, for example that a block ends at the indentation of the *next* line. After that came error recovery, where we measured that the reference reports on average more than two errors for one real mistake.

→ Doc 1, C4, C13, C18 · slide 3

Q2 Walk us through what happens to a program, from text to JSON.

The driver calls `StudentParser.process`, which creates our lexer and parser. CUP pulls tokens from the lexer one at a time: in state YYINITIAL we handle indentation, and in CODE the ordinary tokens. The LALR tables shift and reduce. On every reduction our action code builds the AST node with its location. The `Program` node holds the `Errors` list, and Jackson in the provided driver turns the tree into JSON.

→ Team Prep Pack §2

Q3 What did you change in the provided starter code, and why?

We left the token-factory code and `parseProgram` as they were. We changed two things in the parser code: `syntax_error` asks the lexer for the text of the *specific* token instead of `yytext()`, and we added `enableDefaultReductions` with an `init with` hook. We did not touch anything in `chocopy.common` or `pom.xml`, as the spec requires.

Q4 How do you know your parser is correct beyond the 33 samples?

Differential testing. Our generator writes random valid programs (mixed tabs/spaces, CR/LF/CRLF, comments, missing final newline) and a mutated copy of each. Our tester compares our output with the reference using the grader's rule: the full AST for valid programs, the first error's location for invalid ones. 3,511 files in the final run, all identical. The tester found 13 differences that all 33 samples had missed.

→ `tools/fuzz.py`, `tools/difftest.py` · Doc 1, C21

Q5 If you had another week, what would you improve?

Recovery that keeps the body of a statement whose header is broken: today we parse and check it, but drop it from the AST. And clearer error messages, e.g. "unindent does not match any outer level" instead of only "UNRECOGNIZED". That would mean giving up byte-for-byte agreement with the reference's messages, which aren't graded anyway.

B. Indentation

Q6 How do you emit several DEDENTs for one line? A JFlex action can only return one token.

We pop one level, build one DEDENT, and push the line's whitespace back into the input with `yypushback(yylength())`. The next call matches the same whitespace again, compares it with the smaller stack, and emits the next DEDENT. That repeats until the levels are equal. The only state is the stack.

→ `ChocoPy.jflex` `processIndentation()`

Q7 Why not just keep a queue of pending tokens?

That works too, but you have to wrap the generated `next_token()` and keep the queue in step with JFlex's line/column counters. Re-scanning reuses JFlex's own machinery. The positions come out right automatically, and it's about five lines.

Q8 How do you handle a line with zero indentation? Doesn't the whitespace regex match the empty string?

It would, and JFlex can't fire on an empty match. So we match the whitespace **plus the first real character** (`{WhiteSpace}* [^ \t\r\n#]`) and give the character back with `yypushback(1)`. The match is never empty, and `yytext()` is then exactly the indentation.

Q9 How are tabs handled? What about mixing tabs and spaces?

As in the manual: a tab moves to the next multiple of 8. One tab and eight spaces are therefore the same level, and "4 spaces + tab" is 8. We accept mixing, because the manual defines it. Python 3 would raise a `TabError` for ambiguous mixing, but that rule isn't part of ChocoPy.

→ `indentationWidth()` · edge case `indent_tabs_mixed.py`

Q10 What about blank lines and comments, including a comment indented deeper than its block?

They're not logical lines, so they produce no NEWLINE and their indentation is ignored (manual §3.1.4). In YYINITIAL a rule matches `whitespace* comment? linebreak` and discards it, and longest match makes sure it beats the indentation rule. There are two extra variants for the last line of a file without a line break.

→ edge case `blank_and_comment_lines.py`

Q11 What if the very first line is indented?

Its width is larger than the 0 on the stack, so we emit an INDENT. No rule allows an INDENT at the top level, so the parser reports "Parse error near token INDENT" at (1,1), exactly like the reference.

→ `bad_unexpected_indent_first_line.py`

Q12 What happens with a dedent to a level that was never opened?

Levels 0,4,8 and a line at 2. We pop every level larger than 2, one DEDENT each. The stack is then below the line's width, so we emit an `UNRECOGNIZED` token and treat the line as belonging to the enclosing level. The parser reports the error at that line. The reference pops only one level, and its INDENT/DEDENT become unbalanced: it reports a second, bogus error at EOF. That's our bug report #1.

→ demo `5_reference_bug.py`

Q13 What happens at end of file?

The `<<EOF>>` rule runs again on every call. If we're still in CODE, the last line had no line break, so we emit the missing NEWLINE first. Then we emit one DEDENT per call while the stack is above 0, and finally EOF.

Q14 Why does a DEDENT have a location at all, and why that one?

Because the AST takes the end of every compound statement from it. We found the rule in `coverage.py.ast`: an IfStmt that ends at `[19,8]` when line 19 is indented by 8. So the DEDENT spans the leading whitespace of the line that closes the block, `[L,1,L,width]`. We build the token before pushing the whitespace back; otherwise its length would already be 0.

Q15 Why do nested blocks that end at EOF end at different columns?

Because the reference does that. Its EOF DEDENTs sit at columns 1, 2, 3, …, which is a quirk, since that text doesn't exist. The grader compares locations, so we copy it in `eofSymbol()` and documented it as bug #2. Undoing it is one line.

C. Other lexer questions

Q16 How do you tell keywords from identifiers, e.g. `classic` vs `class`?

JFlex picks the longest match, and on a tie the earlier rule. `classic` is longer as an identifier, so it's an identifier. `class` alone is a tie, and the keyword rule comes first.

Q17 Why is the lexer not allowed to report errors? What do you do instead?

The spec requires all errors to come from the parser, in one format and one list. Anything unacceptable becomes an `UNRECOGNIZED` token, which appears in no rule. The parser then reports "Parse error near token UNRECOGNIZED: …" at exactly that position.

Q18 What's the RESERVED token?

The 16 Python keywords that ChocoPy forbids as names but never uses, such as `lambda`, `import`, `async`. They must not lex as identifiers (manual §3.3), and no rule accepts them, so using one is a syntax error. The reference uses the same name.

Q19 How do you handle string escapes, and what's illegal?

One regex finds where the literal ends, and `stringSymbol()` decodes it: `\" \n \t \\` are legal. Any other escape, or a character outside ASCII 32–126 (such as a raw tab), makes the **whole literal** a single UNRECOGNIZED token, as in the reference.

Q20 What's IDSTRING, and why is it needed?

A string whose contents look like an identifier, e.g. `"Node"`. In type annotations only IDSTRING is allowed (`self: "Node"`, used for classes that aren't defined yet). As a value, both kinds are simply a StringLiteral.

Q21 What about an unterminated string?

If there's no closing quote anywhere later, the lone `"` becomes UNRECOGNIZED and lexing continues right after it, as in the reference. If a quote *does* follow later, the reference's match runs across lines up to that quote. We copy that and report it as bug #3.

Q22 How do you handle 2147483648? And how would someone write −2147483648?

We parse with `Integer.parseInt`. Overflow throws, and the literal becomes UNRECOGNIZED. The minimum int *can't* be written directly, because the minus is a separate unary operator applied to an out-of-range literal. That follows from manual §3.4.2, and the reference agrees.

Q23 Why is `007` an error, and where exactly?

The integer rule is `0 | [1-9][0-9]*`. Longest match splits `007` into `0`, `0`, `7`. Two integers in a row is a syntax error, reported at the second `0`, the same column as the reference.

Q24 Your first string regex didn't compile. Why?

We wrote the character class with `\u005B` for `[`. JFlex replaces `\uXXXX` escapes **before** parsing the regex (like javac), so it became a bare `[` inside the class: a syntax error. We now write the characters escaped: `[ !#-\[\]-~]`.

D. Grammar and precedence

Q25 How did you implement operator precedence?

With CUP `precedence` declarations that follow the manual's §4.1 table, rather than one nonterminal per level. In a shift/reduce situation CUP compares the rule's precedence (its last terminal) with the lookahead's: the higher one wins. On a tie, `left` reduces, `right` shifts, and `nonassoc` is an error.

→ `ChocoPy.cup` precedence block

Q26 Why is `True == not False` a syntax error?

Because of the manual's `expr` / `cexpr` split, which we kept. `==` only combines two `cexpr`s, and `not …` is an `expr`. So after `==`, a `not` can't start an operand. The manual asks for exactly this. You have to write `True == (not False)`.

Q27 How is `a < b < c` rejected?

The comparisons are `nonassoc`. In the state `cexpr < cexpr · <` both have the same precedence, and nonassoc turns that into an error entry in the table. The error is reported at the second `<`.

Q28 What's `%prec UMINUS` for?

The minus token is shared by binary and unary minus, but unary minus binds tighter than `*`. `UMINUS` is a dummy terminal the lexer never produces. `%prec UMINUS` gives the rule `MINUS cexpr` that terminal's higher precedence, so `-a * b` is `(-a) * b`. Because `.` and `[ ]` are higher still, `-a.b` is `-(a.b)`.

Q29 How did you deal with the dangling-else problem?

For statements it doesn't exist: blocks are closed by DEDENT, so an `else` always belongs to the `if` at its indentation. For the conditional expression, `IF` / `ELSE` are declared `right` with the lowest precedence, so `a if b else c if d else e` nests to the right, as in Python.

Q30 How many conflicts does your grammar have? How do you know?

Zero. CUP prints every shift/reduce and reduce/reduce conflict, and since our `pom.xml` has no `expect` option, even one conflict would fail the build. The only warnings are the two terminals that are deliberately unused: `RESERVED` and `UNRECOGNIZED`.

Q31 A variable definition and a statement can both start with an identifier. Is that a problem for LALR(1)?

No. After shifting the identifier, one token of lookahead decides: `:` means a typed variable (declaration), and anything else means an expression or target. The rule "declarations before statements" comes out of the grammar's shape: `program ::= top_decls stmt_list`.

Q32 How do chained assignments like `a = b.c = d[0] = 1` work?

`assign_targets ::= target EQ | assign_targets target EQ`, where a target is an identifier, member expression or index expression. After such an expression, a following `=` reduces it to a target. Because `=` (EQ) never follows an expression anywhere else, there's no conflict. The AssignStmt gets all targets in order.

Q33 How is `elif` represented in the AST?

It isn't. It's syntactic sugar: `if a: X elif b: Y else: Z` becomes `IfStmt(a, X, [IfStmt(b, Y, Z)])`. The right-recursive `elif_else` rule returns a one-element list containing the nested IfStmt, so the chain nests naturally.

→ demo step 3

Q34 You use left recursion for lists, but right recursion for elif. Why?

Left recursion keeps the LR stack small: each element is reduced immediately. Right recursion pushes everything first. For elif, right recursion is what we *want*, because the tree must nest to the right, and elif chains are short.

Q35 Where is `pass` in the AST? And a function without `-> type`?

`pass` builds no node: the action returns `null`, and our list helpers skip nulls, as in the reference. For a missing return type, the AST still gets `ClassType "<None>"`, located at the header's colon. We learned that by probing the reference.

Q36 Why is `f()(1)` a syntax error?

In ChocoPy's grammar a call is `ID ( args )` or `member_expr ( args )`. Only a name or a method can be called, not an arbitrary expression. So after `f()`, a `(` is invalid.

E. Source locations

Q37 How do you compute a node's location?

From the leftmost symbol's left edge to the rightmost symbol's right edge. CUP provides `xleft` / `xright` for every labelled symbol. For a nonterminal, that's the span of all its tokens.

Q38 Why does an `if` statement end at the column of the next line's indentation?

Its last symbol is the block, the block's last token is the DEDENT, and the DEDENT spans the next line's leading whitespace. So `[16,13,19,8]` in coverage.py: the next line is indented by 8.

Q39 In coverage.py a method ends at [34,22] but its class at [36,0]. Why?

Node location vs symbol location. The `FuncDef` node ends at its body, the last NEWLINE. The `func_def` *grammar symbol* also includes the closing DEDENT on line 36, which has zero indentation, so column 0. The class ends at its last member's *symbol*, hence [36,0].

Q40 Which location bug did the fuzzer find?

Programs that start with `pass`, and `else: pass`. `pass` creates no node, and we had computed locations from nodes and from whether a list was empty. The reference uses grammar symbols. We split the rules (if with or without an else part) so locations always come from symbols.

Q41 What's the location of a whole program, and of an empty one?

From the first declaration or statement to the last token consumed, which is a NEWLINE or a DEDENT. An empty file is `[0,0,0,0]`, because Node uses 0 for a null location.

F. Error handling and recovery

Q42 Explain how CUP recovers from an error.

It reports the error, then pops states until it finds one that can shift the special `error` token, and shifts it. Then it discards input tokens until the next three (`error_sync_size`) can be parsed, and continues. If it reaches EOF before that works, it gives up on the rest of the file.

Q43 Why do you have separate statement lists (`stmt_list` and `block_stmts`)?

To avoid a reduce/reduce conflict. At the top level, right after the declarations, both a declaration and a statement can begin. If both had an `error` rule live there, shifting `error` would lead to a state with two completed items. So the top-level statement list only accepts errors *after* its first statement. Inside blocks and function bodies nothing competes, so `block_stmts` may even start with `error`.

Q44 What are your "anchored" recovery rules?

Error rules with a token after `error`, which tells CUP where to continue. A bad initial value (`x:int = 1 + 2`) skips to the line's NEWLINE and stays in declaration mode. A broken `def` / `class` / `if` / `elif` / `while` / `for` header skips to its NEWLINE, and if an indented body follows, it's parsed too, so its INDENT doesn't cause more errors.

Q45 Why anchor on NEWLINE and not on the colon?

We tried the colon first. If the anchor never shows up, CUP searches to EOF and **gives up on the whole file**. Our metric caught this. A missing colon, or an error *after* the block that fell back into the header rule, made the parse abort. A NEWLINE always comes soon. We also moved the normal headers into small nonterminals (`if_header` …): once a header is reduced, its states leave the stack, so a later error can't fall back into it.

Q46 What are default reductions, and why did you add them?

A state is "consistent" if its only possible action is one reduction. We do that reduction on *any* lookahead. Without it, in `z = 5` followed by `lambda = 5`, CUP needed a valid lookahead to reduce `z = 5 NEWLINE` to a statement. It saw RESERVED first, reported the error, and recovery popped the half-reduced `z = 5`, so a *correct* statement vanished from the AST. The reference has the same problem. Bison does this by default; CUP has a `-compact_red` flag, but that's set in `pom.xml`, which we may not edit, so we patch the action table when the parser starts.

→ `enableDefaultReductions()` in `ChocoPy.cup` parser code

Q47 Can your recovery rules or default reductions change where the first error is reported? That's what's graded.

No. `error` never appears in the real input, so adding error rules doesn't change which token sequences are valid. An LR parser reports the first token that can't continue a valid prefix. Default reductions only perform reductions the parser would have done anyway; reductions consume no input, so the same token is rejected. We also checked it empirically: every mutated program has the same first error as the reference.

Q48 How did you measure recovery quality?

`tools/recovery_quality.py` puts exactly one `$` into a random line of each of a few hundred valid programs. An ideal parser then reports exactly one error, keeps all the undamaged code, and never gives up. We count those three things for us and for the reference (see the table below), and kept only changes that improved the numbers.

Q49 Why does `lambda = 5` sometimes go unreported?

If it comes directly after another broken line, it falls inside CUP's 3-token re-sync window and is discarded. We tried windows of 2 and 1: recovery got *worse* (more cascades), so we kept 3. In `bad.py` every case is followed by a correct line, so each mistake gets its own error.

Q50 Your error messages aren't identical to the reference's. Is that a problem?

The spec says only the location of the first error is compared. The format "Parse error near token X: text" is the same. The reference adds hints like `<bad escape>`. We use the token's own source text (stored per token) instead of `yytext()`, which would be wrong after a pushback or at EOF.

Q51 Why an `IdentityHashMap` for the token text?

We want one entry per token *object*. Identity is exactly that, and it doesn't depend on whether `Symbol` overrides `equals`/`hashCode`. The text is attached to the token itself, so it stays correct even while CUP reads ahead during recovery.

G. Testing

Q52 How do you know your generated "valid" programs are really valid?

We check them with the reference. The first time, only 193 of 400 were! The generator built targets like `-x.foo = 1` and chained comparisons hidden inside arithmetic. After fixing it, 500 of 500 were valid. Lesson: test the tester.

Q53 What's in your own test suite?

44 hand-written cases in `src/test/data/pa1/edge_cases` with expected `.ast` files generated by the reference: tabs, CR-only and CRLF line endings, no final newline, empty file, EOF with many dedents, escapes, IDSTRING types, max int, precedence, chained targets, and 20 bad programs. They run with the normal `--test` command.

Q54 What does good.py cover? What does bad.py show?

`good.py` uses nearly every rule: all literal kinds and types, nested functions with global/nonlocal, classes with inheritance and a pass-only class, if/elif/else, while, for, chained and member/index assignments, every operator, conditional expressions, method chains, and blocks that end at EOF. `bad.py` has 9 deliberate mistakes (declarations, headers, statements, indentation, strings, integers) and gets exactly one error each, on distinct lines.

Q55 How long does your test run take?

Each comparison starts two JVMs, about 0.5 s each, so the tester runs comparisons in parallel threads: 1,000 files take about two minutes on a laptop. The 33 samples alone take a few seconds.

H. Team, process, and what comes next

Q56 Who did what? And can each of you explain the other parts?

Mithulram: the lexer and indentation. Nafeel: the grammar, AST and locations. Dinodi: error recovery, the test tools and the bug report. We reviewed each other's code, and each of us has walked through all of it. (Then *show* it: answer the question even when it's about a teammate's area.)

Q57 How did you make sure the code is correct and that you understand all of it?

We didn't trust anything without evidence. Every design decision was checked against the reference with the differential tester and the recovery metric, and several first ideas were thrown away because the numbers said so (the STRING state, the colon anchors, smaller sync windows). We went through every line together.

Q58 What did you learn that you didn't expect?

That "the grammar is right" and "the output is right" are very different things. Most of the work was in locations and error recovery, not in the grammar rules. And that measuring is worth it: two of our "improvements" made things worse, and we only found out from the numbers.

Q59 What will PA2 (semantic analysis) need from your AST?

Correct node types and accurate locations, because semantic errors are reported at node locations. Declarations are kept separate from statements, and type annotations are kept as ClassType/ListType. PA2 only runs on programs without syntax errors, so our recovery nodes (dropped statements) never reach it.

Q60 What is the difference between LALR(1) and LR(1)? Did it matter here?

LALR(1) merges LR(1) states that have the same items but different lookaheads. The tables are much smaller, but in rare cases the merge creates reduce/reduce conflicts that LR(1) wouldn't have. Our grammar has no conflicts, so it didn't matter. LALR may also do a few extra reductions before it detects an error, but it never shifts an erroneous token, so the error position is the same.

Recovery numbers to quote (final version)

One random `$` inserted into each valid program; two independent corpora. "Aborted" means CUP gave up on the rest of the file. **One-liner to remember:** "For one real mistake we report 1.4 errors and the reference 2.0. We report exactly one error three times out of four (the reference: fewer than half), and we never give up on a file."

Corpus	Parser	Exactly one error	Avg. errors per mistake	Undamaged code kept	Aborted parses
A (133 programs)	Reference	43.6 %	2.28	95.6 %	1
	Ours	**76.7 %**	**1.35**	**95.8 %**	**0**
B (213 programs)	Reference	46.0 %	1.84	96.1 %	2
	Ours	**73.2 %**	**1.38**	**96.2 %**	**0**
Both (346)	Reference → **Ours**	45.1 % → **74.6 %**	2.01 → **1.37**	≈ 96 % → ≈ 96 %	3 → **0**